QueryAPIGate HOW-TO GUIDE 52 · ACCESS

Give a partner a quota and a concurrency cap

Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=52-give-a-partner-a-quota-and-a-concurrency-cap

Time: 15 minutes. **You'll end up with:** a partner's key that can run at most two queries at once, get at most 100 rows a call, and make 1,000 calls a day - enforced across every instance, with the partner told how much is left.

> **Stable** - quotas, like the three limits, are under the compatibility promise since 0.30 (what that means).

Every command and response below was run against two QueryAPIGate instances (gunicorn) sharing a PostgreSQL store and Redis, with a PostgreSQL `orders` table behind two saved queries: `orders_by_customer`, and `slow_report` (`SELECT pg_sleep(2), count(*) ...`).

Why

A rate limit counts calls, not their cost. A partner under `60/minute` can still run twenty two-second reports at once and hold every connection the pool has, while everyone else on that connection waits. And contracts are written in quotas - "1,000 calls a day" - which a per-minute rate only approximates.

Step 1: The key

shell

```
curl -X POST http://127.0.0.1:5091/api/v1/api-keys -H "X-API-Key: $TOKEN" -H 'Content-Type:
application/json' -d '{
  "name": "partner-acme", "connections": [], "queries": ["orders_by_customer", "slow_report"],
  "max_concurrent": 2, "max_rows": 100, "quota": {"calls": 1000, "per": "day"}
}'
```

json

```
"effective_limits": {"max_concurrent": 2, "query_timeout": 30.0,
                     "max_rows": {"page": 100, "stream": 100, "mcp": 100}},
"usage_this_period": {"per": "day", "period_start": "2026-10-11 00:00:00", "resets_at": "2026-10-12
00:00:00",
                      "calls": 0, "rows": 0, "calls_limit": 1000, "rows_limit": null}
```

`effective_limits` is what each comes to against the server's own limits - a key's limit only ever narrows them. In the Console: **API keys** → **New** (or Edit) → **Queries at once**, **Time limit**, **Rows per call**, and **Quota**. A role takes the same fields; for signed-in users each gets the role's quota as their own.

Step 2: Two queries at once - and no more

Three slow reports at once, the third on the other instance:

```text
port 5091 → 200 in 2.02s
port 5090 → 200 in 2.05s
port 5091 → 429 in 0.017s
```

```json
{"error": "This caller already has 2 queries running, its max_concurrent - retry when one finishes",
 "code": "too_many_concurrent", "retry_after": 1, "max_concurrent": 2}
```

The count is in Redis, so it holds across instances. A query holds its slot from when it's sent until its last row is read - a stream or a big export for as long as it runs - and each slot has a lease, so one held by an instance that crashed is freed when its lease ends. The third call isn't queued: `Retry-After: 1`, and the client decides.

Step 3: At most 100 rows a call

```shell
curl -sD - -o /dev/null 'http://127.0.0.1:5090/q/orders_by_customer?customer=acme&page_size=500' -H "X-API-Key: $KEY"
# X-Page-Size: 100
# X-Row-Count: 100
# X-Has-More: true
```

Pages are 100 rows; a stream stops at 100; an agent's MCP result is cut to 100 and marked `truncated`.

Step 4: The quota

Every response tells the partner where they stand - here on a `trial` key with 2 calls a month:

```text
HTTP/1.1 200 OK                      HTTP/1.1 200 OK                      HTTP/1.1 429 TOO MANY REQUESTS
X-Quota-Remaining-Calls: 1           X-Quota-Remaining-Calls: 0           Retry-After: 1734669
X-Quota-Reset: 2026-11-01 00:00:00   X-Quota-Reset: 2026-11-01 00:00:00   X-Quota-Reset: 2026-11-01 00:00:00
```

```json
{"error": "This caller's monthly quota of 2 calls is used up - it resets at 2026-11-01 00:00:00",
 "code": "quota_exceeded", "quota": "calls", "limit": 2, "resets_at": "2026-11-01 00:00:00"}
```

The two calls went to different instances; the third was refused on the first. What counts: successful calls that return data and their rows - saved queries, ad-hoc SQL, MCP tool calls, cached responses. A refused or failed call doesn't count; neither did the 429 in Step 2 (`partner-acme` was at 997 left after it).

How exact it is

Two instances, clients calling as fast as they could until refused, a quota of 1,000 calls a day:

	CALLS ANSWERED	OVER THE QUOTA
16 clients, with Redis	1,007	7
32 clients, with Redis	1,016	16
16 clients, **without** Redis	1,615	615

With Redis, the overshoot is the calls in flight at the moment the quota ran out: checked before a call, counted after it. Without Redis each instance sees the others' counts only once they're written to the store and re-read - about every second - so a burst shorter than that gets nearly a whole quota through each instance. **Run several instances with Redis** if a quota is in a contract. One instance alone counts exactly.

Watching it

- **The key:** `usage_this_period`, and in the Console a bar under Usage - amber from 80%, red when used up.
- **Everyone:** `GET /api/v1/usage/quotas` - each caller's calls and rows this day and this month, for your own metering or invoicing (QueryAPIGate doesn't bill):

```json
{"items": [{"caller": "partner-acme", "per": "day", "period": "2026-10-11", "calls": 5, "rows": 104, ...},
           {"caller": "trial", "per": "month", "period": "2026-10", "calls": 2, "rows": 20, ...}]}
```

- **Alerts:** `quota_nearly_used` - *API key 'trial' has used all of its monthly quota of calls* (critical; a warning at 80%).

Next steps

- Rate-limit a key, or the whole server - calls per minute, alongside these.
- Give an external partner access to exactly one query.
- Per-key limits reference and Usage quotas.

QueryAPIGate · one governed gate between your databases and everything that asks them for data · self-hosted, Apache-2.0 · © 2026 QueryAPIGate